

Stalin: The Game

Robin Langer

1 September 2026

1 The Unix philosophy, pipes, and CLI tools

In case the reader is a little rusty, we begin with a brief review of the Unix this note leans on: what a process is, how one reaches the world outside itself, how two of them are joined together, and why a command-line tool has the shape it has. None of this is specific to the game **stalin**, but all of it is used by the game **stalin**. The game comes later, once the concepts are in place.

Each part below introduces one idea and, where it helps, one small example that illustrates only that idea. Nothing is used before it has been introduced.

A process

A *process* is one running program, together with everything the kernel keeps on its behalf: its memory, its position in the code, its identity — which user it runs as — and a number. A program is a file sitting on a disk, and does nothing; a process is that file in motion. Run the same program twice and you have two processes which share nothing and cannot see each other.

The number is the process's *pid*. It is how anything outside the process refers to it, and there is no other handle: to stop a program you send a signal to its pid.

User space and kernel space

A running machine is divided in two. Your code executes in *user space*, where it may compute freely and touch nothing: it cannot address a disk, a network card, or another process's memory, because the processor is in a mode that forbids it. The *kernel* executes in kernel space, where it may touch everything. The division is enforced by the hardware, not by convention.

There is exactly one interface between the two: the *system call*. It suspends your process, switches the processor into kernel mode, runs kernel code on your process's behalf, and returns. Every effect a program has on the world outside its own memory is a system call. There is no other mechanism.

read and write

That interface could have been wide. It is in fact very narrow. The two calls that matter most are these:

```
read (n, buffer, count)  -- give me up to count bytes from the thing numbered n
write(n, buffer, count)  -- take these count bytes and put them in the thing
                          numbered n
```

Each takes a number identifying something already open, a place to put bytes or take them from, and how many. Between them they are very nearly the whole interface to the outside world. What that number is, and where it comes from, is the subject of two subsections below.

Everything is a file

Why should two calls be enough? Because of a design decision usually stated as a slogan: **everything is a file**. A file on a disk, a terminal, a network connection, a source of random bytes — all of them are presented to user space through the same interface, so the same two calls reach all of them.

The slogan overstates itself in one respect, and it is worth being precise about which. Things do differ in how they are *made*: a file is opened by name, a network connection has to be built by a short sequence of calls of its own. What they do not differ in is how they are *used*. Once the thing exists and you are holding its number, **read** and **write** are the whole vocabulary. The uniformity is at the point of use — which is the point that matters, because using is where one program meets another.

File descriptors

That number is a *file descriptor*. It is a small non-negative integer, and it is an index into a table the kernel keeps for each process, whose entries point at the things that process has open. Nothing in the number says what kind of thing it is. The descriptor is an opaque handle, and that is exactly why one program can be pointed at a file one day and at a network connection the next without being rewritten.

Three entries are filled in before a process begins, and it did not fill them in itself — whoever started it did.

0	standard input	the stream the program reads
1	standard output	where its results go
2	standard error	where its complaints go

Start a program from a terminal and all three point at that terminal, which is why results and complaints appear jumbled together in one scrolling window. That is a property of the terminal, not of the program.

cat

Everything so far is enough to write a real Unix program. **cat** — named for *concatenate* — copies its input to its output, unchanged, and does nothing else. That is its entire specification, and it is almost exactly **read** and **write** in a loop:

```
char buf[4096];
for (;;) {
    n = read(0, buf, 4096);    -- take up to 4096 bytes from standard input
    if (n == 0) break;        -- 0 bytes means end of input; stop
    if (n < 0) exit(1);       -- negative means the read itself failed
    write(1, buf, n);         -- put exactly those n bytes on standard output
}
```

The real **cat** is longer, but the extra length is all flags and error messages; that loop is the program. Look at what it does *not* contain. It never asks what 0 is attached to, or what 1 is attached to, because **read** and **write** provide no way to ask. A disk file, a keyboard, another program's output: the loop is identical and the program cannot tell the difference.

fork and exec

Two more system calls, and they are the two that make new processes.

fork *duplicates* the calling process. Where there was one process there are now two, identical in their memory, their open descriptors and their position in the code, differing only in the value **fork** handed back — which is how each tells whether it is the original or the copy.

exec *replaces* the program running inside a process. Same process, same pid, same descriptor table, but the old code and memory are discarded and a new program is loaded in their place.

Neither is a “run this program” call, and that is the point: to start a program, a process **forks**, and then the copy **execs**. The descriptor table survives both, so the copy inherits the original’s descriptors — and in the gap between the two calls the copy is still running the original’s code, so it can rearrange its own table before replacing itself. Three subsections below turn on that gap.

The shell

The shell is not part of the kernel and has no privilege you do not have. It is an ordinary program running in user space with your identity, and its purpose is to start other programs. That is all it is: a program whose job is running programs. Two things it relies on come first, and then what it does with a line you have typed.

The environment

One more thing a process carries: its *environment*, a set of named strings held alongside its memory and its descriptor table. They are called variables, but they are only ever text. Like the descriptor table, the environment survives **fork** and **exec**, so a new process starts with a copy of the one that started it:

```
$ export GREETING=hello      -- add it to this shell environment
$ sh -c 'echo $GREETING'    -- a different process, started by this one
hello
```

That is the whole of it: named text, inherited downwards. The most used of them is **PATH**, which holds a list of directories; the shell searches them whenever it is looking for a program to run.

Globbering

A word containing *****, **?** or **[...]** is a *glob*: a pattern to be matched against the names of files that exist. ***** stands for any run of characters, **?** for a single one, **[abc]** for one character out of a set.

The matching is done by the shell, before any program is started:

```
$ echo *.txt
log.txt notes.txt other.txt
```

echo prints its arguments and nothing else, so what it printed is exactly what it was given: three filenames. It never saw an asterisk. That is why every program accepts patterns, and why no program contains any code for matching them.

What the shell does with a line

Given a line, the shell does five things, and every one of them has already been described.

1. **Split the line into words.** The first word names what to run; the rest become the argument vector that program will receive.
2. **Find the program.** The first word is looked for in each directory named by `PATH`, in order, and the first match wins.
3. **fork.** The shell duplicates itself.
4. **exec.** The copy replaces itself with the program. The shell that read the line is still there, waiting for it to finish.
5. **Keep the exit status.** When the program finishes the kernel gives the shell a small number it left behind, by convention zero for success.

One command cannot work that way, and the reason is worth seeing. `cd` has to be built into the shell rather than being a program, because a program runs in the copy, and the copy changing its own working directory would leave the shell exactly where it was. Anything that must alter the shell's own state has to be part of the shell.

A remark, because this is the mechanism behind something that catches people out with coding agents. Claude Code runs each shell command exactly as the shell runs a program: it forks, and the copy execs. A `cd` in that command changes the copy's working directory, and the copy then exits. So no command the agent runs can change the agent's. The directory it was launched in is therefore the directory it has for the whole session, which is why you must start it in the project you actually mean to work on.

Redirection

Now the gap between `fork` and `exec`. Writing `> log` after a command tells the shell: in the copy, and before exec'ing, open the file `log` and put it in slot 1.

```
$ echo hello > log      -- slot 1 is now the file, not the terminal
$ cat log
hello
```

`2> errs` redirects descriptor 2 in the same way. Nothing was added to `echo` to make this work, and `echo` cannot discover that it happened: it wrote to descriptor 1 exactly as it always does. Redirection is the shell's syntax and the shell's doing, from beginning to end.

Pipes

A *pipe* is a buffer the kernel will hand out on request, with two descriptors: one to write into, one to read out of. `|` is the same redirection as `>`, with a pipe where the file was. The shell asks for a pipe, forks twice, puts the writing end in the left program's slot 1 and the reading end in the right program's slot 0, and execs both.

```
$ cat notes.txt | wc -l
3
```

`wc -l` counts the lines on its standard input. It was given no filename and does not know one exists.

Both programs run at the same time; `wc` does not wait for `cat` to finish. The buffer is finite — 64×1024 bytes on Linux — and the kernel makes each side wait for the other rather than growing it. When it is full, `cat`'s next `write` does not return until `wc` has consumed something; when it is empty, `wc`'s next `read` does not return until `cat` has produced something. Neither program contains a line of code about this.

Two things follow, and both are worth having in mind. A pipeline over a very large file needs no more memory than a pipeline over a small one, because at no moment does either process hold more than the buffer. And the second program starts answering immediately rather than at the end: in `cat huge.log | grep x`, the command `grep` is handed the first 64×1024 bytes as soon as `cat` has written them and reports its first match from those, while `cat` is still reading. It does not need to wait for `cat` to finish, and nothing in either program arranges for it not to. That is simply what two processes and one finite buffer do.

Exit status, and the shell's control flow

The small number a program leaves behind is what the shell builds its control flow on.

- `;` — run the next command regardless of the status.
- `&&` — run the next command only if the previous one exited zero.
- `||` — run the next command only if the previous one exited non-zero.
- `&` — start this command and do not wait for it, so several can run at once.

We could say a good deal more about all this, but it is not really relevant to the game `stalin`.

Shell scripts

A file of such lines is a shell script, and running it is the same as having typed them. So a pipeline that worked once can be kept and named. And because a script is started like any other program — found on `PATH`, forked, `execed`, its status collected — a script can appear in a pipeline itself, on either side, and nothing that uses it needs to know it is a script.

Three channels, not one

Gather up what a program actually offers the world outside it. There are three separate channels, and they carry three different kinds of thing.

Channel	Carries	Read by
descriptor 1 (stdout)	the results	the next program in the pipeline
descriptor 2 (stderr)	the narration	a person
the exit status	the verdict	the shell

Keeping them apart is what makes a program usable by another program: a pipe takes descriptor 1 and nothing else, so whatever reads from a program receives its results and never has to sort them out from error text. Putting results and narration both on descriptor 1 is the commonest way to make a program unusable in a pipeline.

Manual pages, and what is not checked

None of the above is checked by anything. The interface between two programs is bytes: any two can be joined, which is the strength, and nothing verifies that the composition means anything, which is the cost. What a program promises is written in its manual page and nowhere else, so the only thing enforcing it is that you read it.

Here is what that costs. You want to build only if the log has no errors in it, so you write this:

```
$ grep -c ERROR log && ./build
```

`&&` runs the next command only if the previous one exited zero, and `grep` exits zero when it *finds* something. So the line does the exact opposite of what you wanted.

```
$ grep -c ERROR dirty.log && ./build
1
    building                -- errors present, grep exits 0, so it builds
$ grep -c ERROR clean.log && ./build
0
    -- no errors, grep exits 1, so it does not
```

It builds exactly when the log is dirty and refuses exactly when the log is clean. Both halves of `grep`'s behaviour are documented and neither is enforced, the shell did precisely what it was told, and nothing anywhere reports a problem.

What you wanted was `grep` answering *did you find anything* rather than *how many*. That is what `-q` is for: it prints nothing at all and reports only in its exit status — zero if it found at least one match, one if it found none. Put the test where the shell can actually see it, and use `||` so the build runs when nothing was found:

```
$ grep -q ERROR log || ./build
```

Notice that `-c` was carrying that same exit status all along. The count it printed was never the thing `&&` was reading.

A note before leaving this. Claude is an absolute master at writing shell scripts, and this is genuinely a place where you do not need the details. Ask Claude for a script that builds only when the log is clean and you will get a correct one. What you do need is the concept.

None of this is enforced, and this is a deliberate design decision. The only thing holding any of it together is “developer discipline” — somebody read the manual page — and at this level of the stack that is the whole of the enforcement there is.

The dictum

Let's summarise what we have: one uniform interface reached by `read` and `write`, three separate channels, and a pipe that joins one program to another. With that on the table, the philosophy reads as a consequence of the machinery rather than as an aspiration. Doug McIlroy, who wrote the pipe into Unix v3 in 1973, put it in three sentences:

Write programs that do one thing and do it well. Write programs to work together.
Write programs to handle text streams, because that is a universal interface.

Mike Gancarz later expanded this into nine tenets — small is beautiful; each program does one thing well; build a prototype as soon as you can; choose portability over efficiency; store data in flat text; use software leverage; use scripts to increase leverage and portability; avoid captive user interfaces; make every program a filter.

The last is the one that matters here, and it is the least obeyed. A *filter* is a program that reads a stream and writes a stream: its input is not a dialogue and its output is not a report to a human. `cat`, `wc` and `grep` are filters, which is why the examples above could be assembled without any of them having been designed for each other.

A CLI tool is already a dependent function

One last observation, and it is the one this note turns on.

A command-line tool receives one prompt — its argument vector — and produces a response whose *type depends on which prompt it was*. `git log` answers with a list of commits; `git status` answers with a working tree. Those are not the same kind of thing, and which kind you get is fixed by the word after `git`. That is $(p : P) \rightarrow Rp$, with P the subcommands and R the result type of each. The interface (P, R) is what `--help` prints.

Notice what is *absent*. No event loop, no callbacks, no waiting, no concurrency. A CLI starts, is prompted once, answers, and exits. It is nonetheless event-driven in the sense that matters, because what makes a program event-driven is the prompt-indexed response type and not the waiting. Strip the loop away and this is what remains.

2 The Claude Code harness

Claude Code is a CLI process. When it runs a command it does what the shell does: it **forks**, and the copy **execs**, and the copy inherits its parent's identity.

That's it. It runs with the permissions of the user that executed it. It can read and write any file that user can read and write, and it can execute any command that user can execute.

In production one would create a user with limited permissions and run the Claude Code executable as that user.

3 HTTP and JSON

Again, in case the reader is a little rusty, we review some concepts from web app development: what a request and a response are, why so little of HTTP turned out to be the right amount, what a status code is for, which part of a request carries what, and what JSON does and does not preserve. As in the first section, none of it is specific to the game **stalin**, but all of it is used by it.

The examples below all run against a toy server that holds a list of books. It has nothing to do with the game; it exists so that each point can be made on its own.

A request and a response

HTTP is a request/response protocol. A client sends a request; a server sends a response; nothing else happens. Here is one complete exchange, with `curl` showing both halves — the lines it sent marked `>` and the lines it received marked `<`:

```
> GET /books/1 HTTP/1.1
> Host: 127.0.0.1:8099
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: application/json
< Content-Length: 63
<
{"id": 1, "title": "Dead Souls", "year": 1842, "inPrint": true}
```

That is the whole protocol in one screen. A request is a *method* (`GET`), a *path* (`/books/1`), some *headers*, and optionally a *body*. A response is a three-digit *status* (`200`), some *headers*, and a

body. Every web interaction is a variation on this, and one page load is many such exchanges.

A design that looked far too weak

It is worth a paragraph on where so little came from, because HTTP made the same kind of bet Unix made, and it looked just as inadequate at the time.

Tim Berners-Lee's first version, at CERN in 1991, had one method. You opened a connection, sent a single line — `GET /thepage` — and read the document that came back. There were no headers, no status codes, and no content types. The end of the response was signalled by the connection closing. That is the whole protocol. Methods, statuses and headers arrived in 1996, and persistent connections in 1997, but the shape did not change: one request, one response, and a server that remembers nothing about you afterwards.

Statelessness in particular looked like a straightforward defect. A protocol that cannot remember the previous request cannot have sessions, cannot have transactions, cannot notify you when something happens, and cannot check that what you sent was even the right kind of thing.

What the weakness bought is exactly what it looks like it costs. Because a request carries everything needed to answer it, *any* server can answer *any* request: put a hundred machines behind one name and requests may be sprayed across them in any order. Because a response depends only on its request, anything in between may keep a copy and answer the next one itself. And because the server holds nothing on your behalf, a server that dies loses one request rather than your session. Load balancing and caching were not added to HTTP later — they are consequences of the thing that looked like the flaw.

The best evidence is what has happened since. The encoding on the wire has been replaced twice — a binary framing in 2015, and a different transport underneath it in 2022 — and both times the semantics were left exactly as they were: a method, a path, some headers, a body, and a three-digit status. Two complete rewrites of how the bytes travel, and nothing above had to be rethought.

The parallel with the first section is close enough to be worth naming. A pipe carries bytes and does not know what they mean; an HTTP response carries bytes and a string saying what they are. In both cases the interface is universal precisely because it is weak, in both cases nothing verifies that an exchange means anything, and in both cases what was rebuilt on top of the weakness had to be rebuilt without any help from the protocol at all.

The method says what kind of thing you are doing

The method is the verb. There are a handful in common use and only the first two appear below.

Method	Means
GET	read something; change nothing
POST	create something
PUT	replace something entirely
PATCH	change part of something
DELETE	remove something

The distinction that matters is the first one. A `GET` is meant to have no effect, which is why anything in the network is free to cache it or repeat it. Asking the toy server for a book and asking it to add one differ in the verb, and the two answers differ accordingly:


```
$ curl -X GET http://127.0.0.1:8099/books/1 -- 200 OK
$ curl -X POST http://127.0.0.1:8099/books \
  -H 'Content-Type: application/json' \
  -d '{"title":"The Overcoat","year":1842}' -- 201 Created
```

The status code says what happened

The three-digit number is the server’s answer to “how did that go”. The first digit is the class, and the class is the part worth knowing.

Class	Means	Whose fault
2xx	it worked	—
3xx	look somewhere else	—
4xx	your request was wrong	the client’s
5xx	my handling of it went wrong	the server’s

The 4xx/5xx split is the whole value of the vocabulary. It lets a caller tell “I asked for something impossible” from “the far end is broken”, and those call for completely different responses: fix the request, or try again later. Four requests to the toy server, and four different answers:

```
200 asked for a book that exists
404 asked for a book that does not
400 sent a body that is not JSON
400 sent JSON with no title
```

Note that the last two are both 400. 404 and 400 are both the client’s fault, and the code alone does not say which detail was wrong — so a useful server puts that in the body:

```
$ curl -X POST -d 'not json' http://127.0.0.1:8099/books
{"error": "body is not JSON"}
$ curl -X POST -d '{"year":1842}' http://127.0.0.1:8099/books
{"error": "title is required"}
```

Headers describe the message

Headers are name/value lines carrying facts about the message rather than the thing being asked for. Two appear in the exchange above.

Content-Type says what the body is: **application/json** here, **text/html** for a web page. It is a string, and it is a claim — nothing checks that a body labelled JSON is JSON. **Content-Length** says how many bytes the body has, so the reader knows where the message ends without waiting for the connection to close.

The useful discipline is the one the names suggest: a header is for what is true of the *message*, and the body is for the thing itself. Anything that is neither belongs in neither.

Where data can travel

There are four places a piece of information can ride in a request, and they are not interchangeable.

Place	Example
the path	<code>/books/1</code> — which thing
the query string	<code>/books?year=1842</code> — how to filter or sort it
a header	<code>Content-Type: application/json</code> — about the message
the body	the JSON document itself

The one rule worth stating outright: never put a secret in a path or a query string. Both are part of the URL, and URLs are written to server logs, kept in browser history, and passed along in referrer headers. A password in a query string will end up in a log file that nobody thought of as sensitive.

Statelessness

Each request stands alone. The server does not inherently remember that you asked something a moment ago, and two identical requests get identical answers.

Everything that appears to contradict this is built on top. When a site remembers you are logged in, what happens is that the server sent a `Set-Cookie` header once and the browser now attaches `Cookie: session=abc123` to every subsequent request. The server looks that value up and says, in effect, “ah, user 42 again”. Being logged in is not a state the protocol knows about; it is a string being re-sent, every single time, by a client that could just as easily send a different one. That last point is why a server must treat what arrives as a claim rather than a fact.

JSON

JSON — JavaScript Object Notation — is a text format for data. It has six types and no others: objects, arrays, strings, numbers, booleans, and `null`.

```
{
  "title": "Dead Souls",      // a string
  "year": 1842,               // a number
  "inPrint": true,            // a boolean
  "tags": ["russian"],        // an array
  "author": { "name": "Gogol" }, // an object
  "sequel": null              // null
}
```

Being text is what makes it useful here: it is exactly what a pipe can carry and what a socket can send, so it crosses every boundary in this note unchanged.

What JSON does not carry

The omissions matter more than the six types, because each one is something that silently fails to survive the crossing.

No functions, and no “absent”. Serialise an object with a function in it, or a field explicitly set to `undefined`, and they are simply not there afterwards. No error is raised:

```
const o = { title: "Dead Souls", year: 1842, tags: ["russian"],
            reissue: undefined, describe: () => "a book" };
JSON.stringify(o)
// {"title":"Dead Souls","year":1842,"tags":["russian"]}
```

One number type. Every number is an IEEE 754 double. That is fine for a year and not fine for money or for identifiers:

```
JSON.stringify({ n: 0.1 + 0.2 }) // {"n":0.30000000000000004}
JSON.stringify({ big: 9007199254740993 }) // {"big":9007199254740992}
```

The second is not a rounding display: the value changed. Past 2^{53} the integers are no longer all representable.

No sum types, so a constructor becomes a convention. There is no notation for “this is one of the following kinds of thing”. What people do instead is agree on a field:

```
JSON.parse('{"tag":"Book","title":"Dead Souls"}').tag
// "Book" -- a string, exactly like any other string
```

Nothing distinguishes that from a title that happens to read "Book", and nothing requires the other fields a Book is supposed to have to be present.

No schema at all. There is no place in the format to say what the fields should be. Both ends may agree, but the agreement lives in prose and in code, exactly as in the first section.

Hence the shape of every crossing, in both directions. What arrives is a text document of unknown structure, and if anything is to be relied upon, something has to look. That is the subject of the next section.

4 TypeScript, event handlers, and servers

One more review, and this one is about what a type system can and cannot do for you at a boundary. The concepts are: what TypeScript checks and when, what a cast really is, how a value of unknown shape is turned into one of known shape, how a finite choice is written down, how a response type can depend on which request arrived, and what a server handler is. The examples continue with the shelf of books; none of them is from the game.

Types are checked before the program runs

TypeScript is a type system over JavaScript. The types are annotations: you write them, a checker reads them, and then they are removed and ordinary JavaScript runs.

```
interface Book { title: string; year: number }
const b: Book = { title: "Dead Souls", year: "1842" };
```

```
$ deno check t1.ts
TS2322 [ERROR]: Type 'string' is not assignable to type 'number'.
const b: Book = { title: "Dead Souls", year: "1842" };
~~~~~
```

Nothing ran. The mistake was found by reading the program, which is the whole point of having a checker, and it is the pleasant half of the story.

A cast is a promise, not a check

The types are *erased*. The program that executes has no representation of them and no check derived from them. So the `as` operator is not a conversion — it is an assertion that the checker accepts without verifying and the runtime cannot verify at all:

```
const raw = '{"title":"Dead Souls","year":"1842"}';
const b = JSON.parse(raw) as Book;    // a promise, not a check
console.log(b.year.toFixed(0));       // compiles: year is declared number
```

```
$ deno check t2.ts      -- passes. Nothing is wrong as far as the checker knows.
$ deno run    t2.ts
TypeError: b.year.toFixed is not a function
```

The checker was told `year` is a number and believed it. The `year` was a string all along, and the error surfaced at the first place that actually used it — which may be a long way from the cast, and in a real program is likely to be somewhere else entirely.

unknown is what you want at a boundary

Two types describe a value whose shape you do not know. `any` switches the checker off: every operation on an `any` is permitted. `unknown` does the opposite — it permits nothing until you have established what you have.

```
const raw: unknown = JSON.parse('{"title":"Dead Souls","year":"1842"}');
console.log(raw.year);
```

```
TS18046 [ERROR]: 'raw' is of type 'unknown'.
```

That refusal is the useful thing. `any` would have compiled and then failed later, in the manner of the previous subsection. `unknown` makes the boundary visible by refusing to let you cross it accidentally.

Validate, do not cast

So how does a value of unknown shape become one of known shape honestly? By code that looks at the fields. The signature to aim for is `unknown` in, and the type *or nothing* out:

```
function asBook(u: unknown): Book | null {
  if (typeof u !== "object" || u === null) return null;
  const o = u as Record<string, unknown>;
  if (typeof o.title !== "string") return null;
  if (typeof o.year  !== "number") return null;
  return { title: o.title, year: o.year };    // rebuilt, not asserted
}
```

```
$ deno run t6.ts
accepted: Dead Souls 1842
refused: {"title":"Dead Souls","year":"1842"}
refused: {"year":1842}
```

Four things are deliberate. The parameter is **unknown**, so nothing can be done to it by accident. **Refusal is a value** — **null**, not an exception — so the caller has to decide what to do about it. The returned object is **constructed**, field by field, rather than asserted, so it is a **Book** because it was built as one. And the checks are what make the type true: after this function, a **Book** really does have a numeric year, and everything downstream may rely on it.

A tagged union writes down a finite choice

When a value is one of several kinds of thing, the kinds are written as a union, each carrying a field that says which it is:

```
type Query =
  | { kind: "count" }
  | { kind: "find"; title: string }
  | { kind: "since"; year: number };
```

The value of writing it this way is that the checker can tell the cases apart. Inside `if (q.kind === "find")` the type of `q` is the second member and `q.title` is available; in the other branches it is not, and asking for it does not compile.

The response type can depend on the request

Now the part that matters for everything above. Each of those three queries deserves a different kind of answer: a count is a number, a search is a book or nothing, a filter is a list. That is a function from the request to the *type* of the response, and over a finite set of cases it can simply be written down:

```
interface Answers { count: number; find: Book | null; since: Book[] }
type Reply<K extends Query["kind"]> = Answers[K];
```

`Reply<"count">` is `number`, `Reply<"since">` is `Book[]`. Read `Answers` as the table of a function from the tag to a type.

The handler table, total by construction

A program answering those queries is a function whose result type depends on its argument. Over a finite tag set it can be written as a record, one entry per case:

```
type Handlers = { [K in Query["kind"]]: (q: Extract<Query, { kind: K }>) => Reply<K> };

const handlers: Handlers = {
  count: (_q) => SHELF.length,
  find: (q) => SHELF.find(b => b.title === q.title) ?? null,
  since: (q) => SHELF.filter(b => b.year >= q.year),
};
```

Each entry is checked against its own case: inside `find`, `q` is the `find` member and the return type is `Book | null`, not a union over all three.

And the table is total, not by anyone's diligence but by the shape of the type. Add a fourth query and the object stops satisfying `Handlers`:

```
TS2741 [ERROR]: Property 'oldest' is missing in type
    '{ count: ...; find: ...; since: ...; }' but required in type 'Handlers'.
```

There is no default branch to fall through and no dispatch to forget. A whole class of mistake becomes unwritable rather than merely untested.

A server handler, and who owns the loop

A server is the same idea with the request arriving over a socket. What you write is a function from a request to a response; the runtime supplies everything else — the socket, the accept loop, the parsing, the concurrency:

```
Deno.serve({ port: 8098 }, (req: Request): Response => {
  if (req.method !== "GET") return json({ error: "GET only" }, 405);
  return json(SHELF, 200);
});
```

```
$ curl -i http://127.0.0.1:8098/
HTTP/1.1 200 OK
content-type: application/json

[{"title":"Dead Souls","year":1842}]

$ curl -X POST http://127.0.0.1:8098/ -- 405
```

That is what “event handler” means operationally: inversion of control, with the loop on the far side of the library boundary. But notice that the loop is not what makes it event-driven in the sense that matters. A command-line tool has no loop at all and is event-driven in the same sense. What both have is a response whose type is indexed by the prompt. The loop is scheduling; the indexing is the type.

Where the type system stops

Put the two halves of this section together and the shape of the whole problem appears.

On the inside, the checker is strong. It computes which answers belong to which requests, makes a handler table total, and turns a class of mistake into a compile error.

On the outside, at the wire, it contributes nothing whatsoever. Everything arriving is `unknown`; the checker ran before the process started and has no representative in it. A single unchecked cast at that boundary forfeits every guarantee inside it, because from then on the checker is reasoning about a claim rather than a fact.

Which is why the whole of it rests on the smallest piece: a function with the signature `unknown → T | null`, that looks at the fields, and refuses.

Summary

One shape recurs at every level of this stack: **a prompt determines the type of its response**. What changes from level to level is only who enforces it.

Level	Prompt	Response	Enforced by
a system call	read/write on a descriptor	bytes, or an error	the kernel
a pipe	the bytes upstream	the bytes downstream	nobody
a CLI tool	the argument vector	stdout and an exit status	a manual page
an HTTP request	method, path, headers, body	a status and a body	the status code
a handler	a tagged request	the answer at that tag	the compiler

The enforcement gets stronger as you read down that table, and it also gets narrower. The kernel checks everything and understands nothing: it will not let you read a file you have no right to, and it has no opinion whatever about whether the bytes mean anything. The compiler understands everything and checks only what never left the process: it knows exactly which answer belongs to which request, and it has gone home before the first byte arrives.

Neither end can be dispensed with, and neither reaches the other. So the engineering is all at the handovers, and there are three of them in this note.

- **From a pipe to a program.** The interface is bytes, so any two programs compose and nothing checks that the composition means anything. What makes a program usable by another program is a discipline nothing enforces: results on descriptor 1, narration on descriptor 2, a verdict in the exit status, and a manual page somebody has read.
- **From a program to a network.** HTTP adds exactly one thing the pipe lacked: a three-digit number saying what happened, and in particular whether the fault was the caller's or the server's. It is a small addition and it is the difference between a caller that can respond sensibly to failure and one that cannot.
- **From the network to the type system,** and this is the important one. What arrives is text of unknown structure. A function of type `unknown → T | null` that looks at the fields, rebuilds the value rather than asserting it, and returns `null` when the value is not what was claimed, is the only thing that makes the guarantees on the inside mean anything at run time. Cast instead, and the compiler spends the rest of the program reasoning carefully about something that was never true.